

Contents

Project Description	4
Iteration 1 – Initial Implementation	5
Iteration 2	6
Modifications.....	7
Collaboration	8
Student 1: Diana Ali Silva.....	8
Diana's Screenshots.....	9
Student 2: Carlos David Urra Cabello.....	13
Carlos's Screenshots.....	13
Problems Encountered.....	17
Student 3: Georgii Taisaev	18
Georgii's Screenshots	18
Student 4: Muslim Muradov - This is from report 2.....	20
Muslim's Screenshots.....	20
Conclusion	22
Link to our repository:	23
References	24



Student name:	Student 1: Diana Ali Silva Student 2: Carlos David Urrea Cabello Student 3: Georgii Taisaev Student 4: Muslim Muradov				
Student number:	Student 1: 3123965 Student 2: 3125350 Student 3: 3119655 Student 4: 3128794				
Faculty:	Computing Science				
Course:	BSCH/BSCO/EXCH		Stage/year:	2	
Subject:	Software Development 2				
Study Mode:	Full time	☒		Part-time	
Lecturer Name:	Gemma Deery				
Assignment Title:	Review 2				
Date due:	30 April 2025				
Date submitted:	30 April 2025				
Plagiarism disclaimer: <i>I understand that plagiarism is a serious offence and have read and understood the college policy on plagiarism. I also understand that I may receive a mark of zero if I have not identified and properly attributed sources which have been used, referred to, or have in any way influenced the preparation of this assignment, or if I have knowingly allowed others to plagiarise my work in this way.</i> <i>I hereby certify that this assignment is my own work, based on my personal study and/or research, and that I have acknowledged all material and sources used in its preparation. I also certify that the assignment has not previously been submitted for assessment and that I have not copied in part or whole or otherwise plagiarised the work of anyone else, including other students.</i> Signed: _____ Date: _____					

Please note: Students **MUST** retain a hard / soft copy of **ALL** assignments as well as a receipt issued and signed by a member of Faculty as proof of submission.

Project Description

The 2D Multiplayer Maze Game is a Java application inspired by the classic arcade games *Amazing Maze* (1976) and *Maze Craze* (1980). The game challenges two players to race through a randomly generated maze to reach the exit first. Each maze is built using a recursive backtracking algorithm to ensure it is always solvable and offers a fresh layout in every round.[1,2]

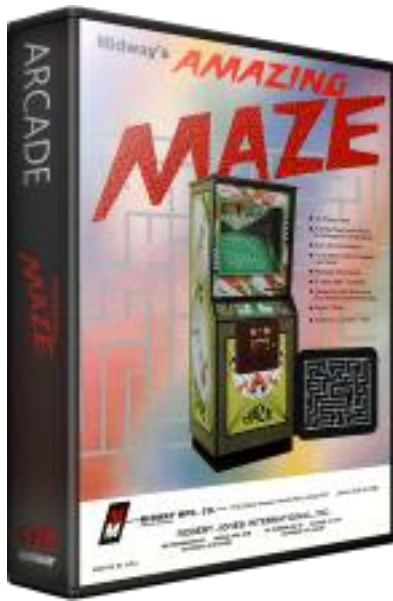


Figure 1. *Amazing Maze* (1976).



Figure 2. *Maze Craze* (1980).

Originally developed as a console-based game using ASCII characters, the 2D *Multiplayer Maze Game* has now entered a new phase focused on transitioning to a graphical version using Java Swing. This iteration introduces a user-friendly menu interface, visual rendering of players, sprites, and maze walls, as well as dynamic sprite movement and collision-based freeze effects. These enhancements aim to improve visual clarity, gameplay feedback, and overall user experience.



Figure 3. Menu of the game

Made by Carlos Urra

Iteration 1 – Initial Implementation

In the first stage of the project, the team focused on building the foundation for the game. We developed core features, established a basic version of the maze, and prepared the structure for future improvements like the graphical interface and multiplayer support.

Carlos set up a GitHub repository to help the team collaborate and manage versions more efficiently. He implemented the maze generator using recursive backtracking, added the basic single-player logic including exit detection, and organized the project folders based on the structure provided in the Moodle example.[3]

Diana implemented a Java Swing window that opened when running the class. At this point, the window displayed only a white screen, but it marked the beginning of transitioning from the console-based version to a graphical user interface.[4]

Georgii added placeholder sprites, represented as 'X' in the console version, which introduced basic challenge elements to the gameplay. These sprites were designed to be replaced with graphic components once Swing was integrated.

Muslim worked on the local multiplayer logic, allowing both players to use the same keyboard to control their characters. Although simultaneous movement was not yet supported due to console limitations, the basic structure was in place for future expansion.

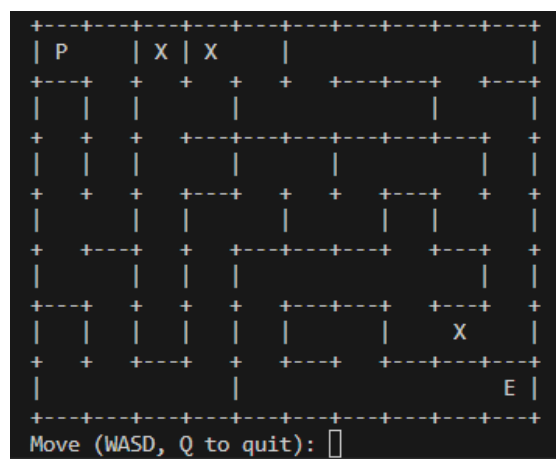


Figure 4. Console-based maze from the previous iteration showing player, exit, and placeholder sprites.

Iteration 2

In Iteration 2, the team began transitioning the game from a console-based format to a graphical environment. The main focus was on laying the groundwork for a future Java Swing interface. This included creating a basic game window, preparing the structure for visual components, and adapting the multiplayer control system.[4]

The maze generator was modified to support mirrored layouts, ensuring both players navigate mazes of equal complexity from opposite starting points. Multiplayer input logic was introduced to allow two players to control characters using separate key sets. While the full graphical interface was not yet implemented, this iteration established the technical base for visual gameplay in the next phase.[3,5]

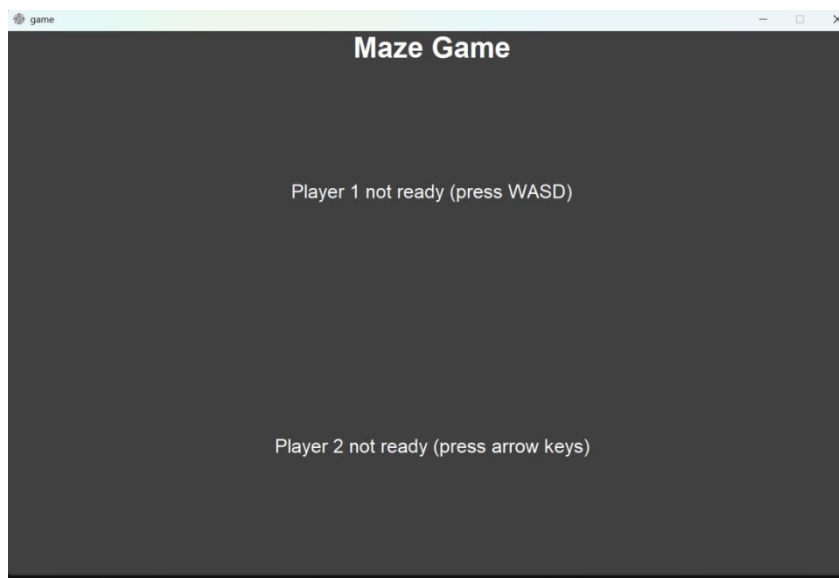


Figure 5. First graphical window for the game

Modifications

The third iteration focused on building the visual framework of the game using Java Swing, transforming it from a logic-driven console program into a GUI-based application.

A new menu interface was introduced as the main entry point for players. This was achieved by creating dedicated Swing components such as the `InstructionPanel` and `ReadyPanel` classes. These panels provide navigation buttons, control guides, and a visual indicator of when players are ready, improving clarity and player onboarding.

The `GameWindow` class was developed to encapsulate the visual environment and coordinate transitions between menu screens and gameplay. It connects the underlying game logic (`MazeGame`) to graphical components and manages layout flow between panels.

To enhance the gameplay experience, new features were integrated:

- A freeze mechanic was added to temporarily disable player movement after sprite collisions, with visual color changes indicating the frozen state.
- The graphical maze, players, and sprites were rendered using shapes and colors within the `MazePanel` class.
- Player collisions and win conditions are now detected and displayed through GUI-based messages.

Throughout this phase, the team collaborated closely using GitHub and regular communication. Tasks were divided based on strengths and progress, and shared code was reviewed, discussed, and tested together. This coordinated effort ensured smooth integration between game logic and graphical features, resulting in a more polished and interactive version of the game.

Collaboration

Student 1: Diana Ali Silva

As the designer for the graphical user interface (GUI) of the maze game, my primary responsibility was to transform the original console-based version into a visually engaging and user-friendly experience. The first major task was designing an intuitive home screen that provides clear navigation options for players. The home screen features a **Start** button to initiate the game and a **Quit** button to exit, ensuring straightforward accessibility. The layout was kept clean and functional to avoid overwhelming the user while maintaining an appealing aesthetic.

Upon clicking the **Start** button, players are directed to an **instruction page**, which outlines the necessary controls before gameplay begins. To ensure both players are prepared, the screen indicates that **Player 1** must confirm readiness using the **WASD keys**, while **Player 2** must use the **arrow keys**. This step prevents confusion and guarantees that both participants are aware of their respective controls before the game commences. The instructions were deliberately concise yet informative to maintain player engagement without unnecessary delays.

Another critical aspect of my role was redesigning the game's visual elements to replace the original console-based graphics with a more polished and immersive style. The **player characters, spirits, walls, and floor tiles** were all updated from simple text symbols to proper **graphical icons**, enhancing the game's overall presentation. The maze walls were given a more defined, textured appearance, while the floor was designed with a consistent visual theme to improve readability and immersion. These changes not only made the game more visually appealing but also helped distinguish key elements, such as enemies and obstacles, at a glance.

By implementing these improvements, the transition from a console-based maze game to a fully graphical interface was successfully achieved. The new design prioritizes usability while maintaining an engaging and cohesive visual style, ensuring players have a seamless and enjoyable experience from the home screen to the gameplay itself.

Diana's Screenshots



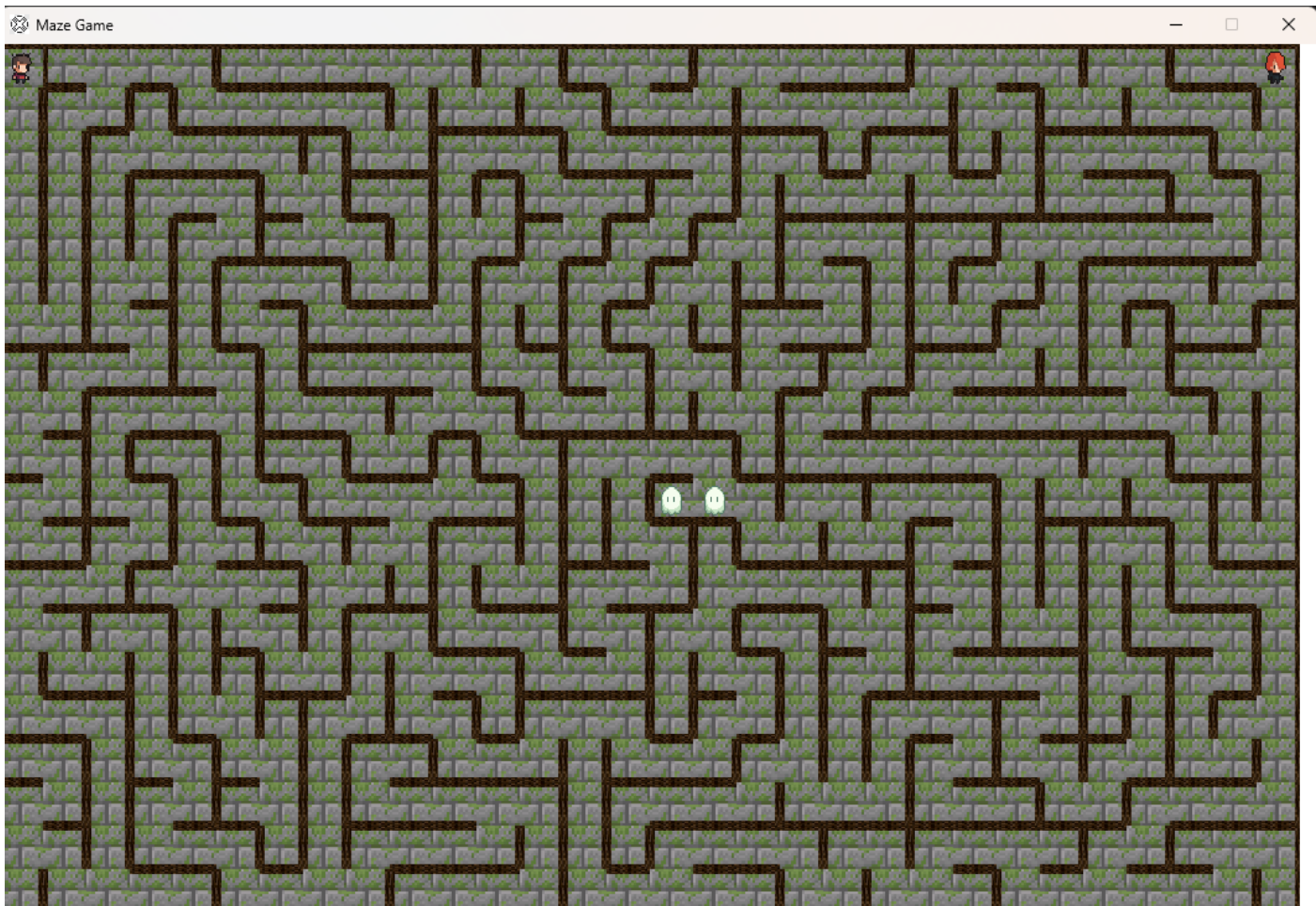
Home Page.



Instructions Page.



Player Ready.



The Maze.



Players 1 and 2.



The spirit.

Student 2: Carlos David Urrea Cabello

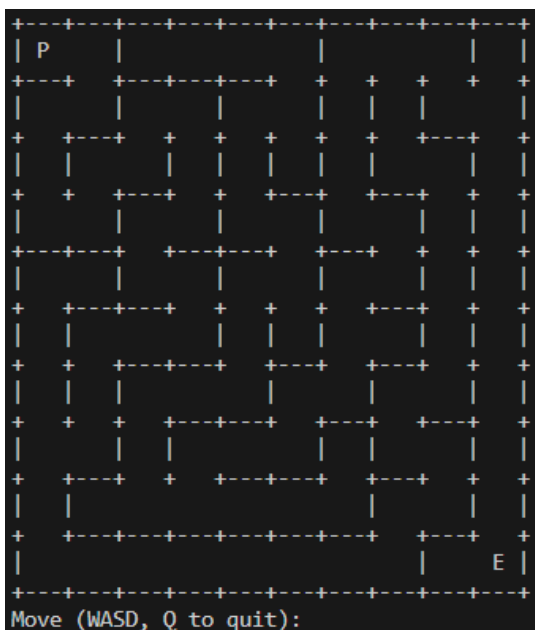
During this iteration, I focused on connecting the existing maze logic to the graphical interface and ensuring smooth gameplay within the new GUI structure. I implemented the MazePanel class, which is responsible for drawing the maze walls, players, and sprites using basic shapes and color-coded indicators. I also introduced the freeze feature, which temporarily disables player movement after colliding with a sprite, and added a visual cue (color change) to represent the frozen state.[4]

I extended the game logic to display winner messages graphically when a player reaches the goal, replacing the older console-based alerts. Additionally, I helped integrate the GUI with the existing player control system, making sure the WASD and IJKL movement mappings were fully functional within the Swing environment. These contributions ensured that the logic from previous iterations was effectively translated into an interactive, visual format.[4]

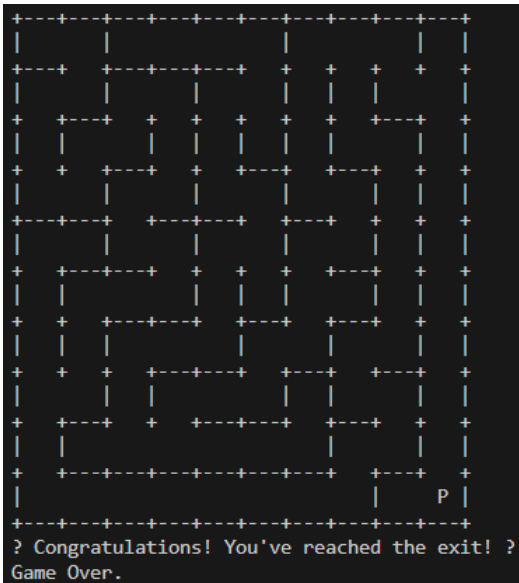
Carlos's Screenshots

[Software_Development_Project](#) PublicJava Updated 3 hours ago

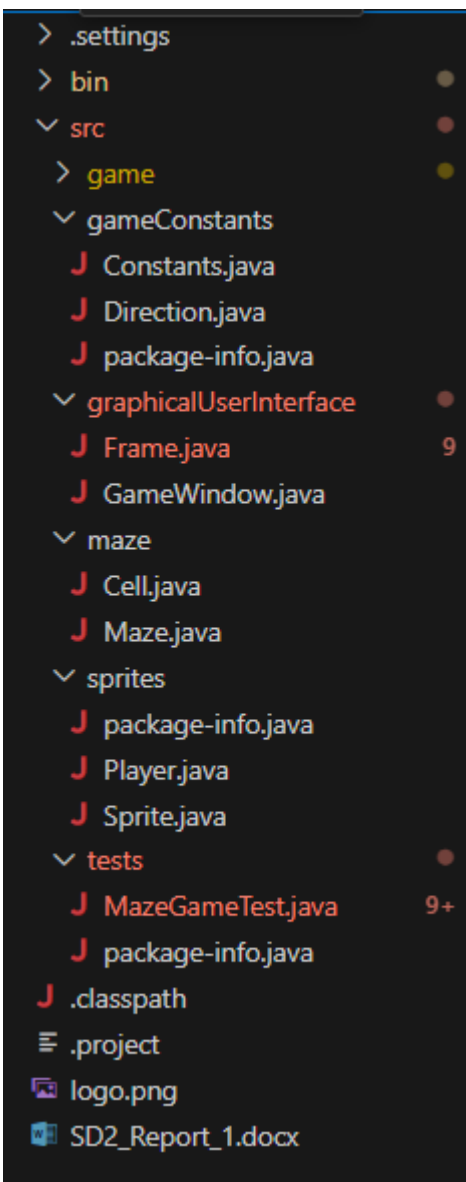
Github repository



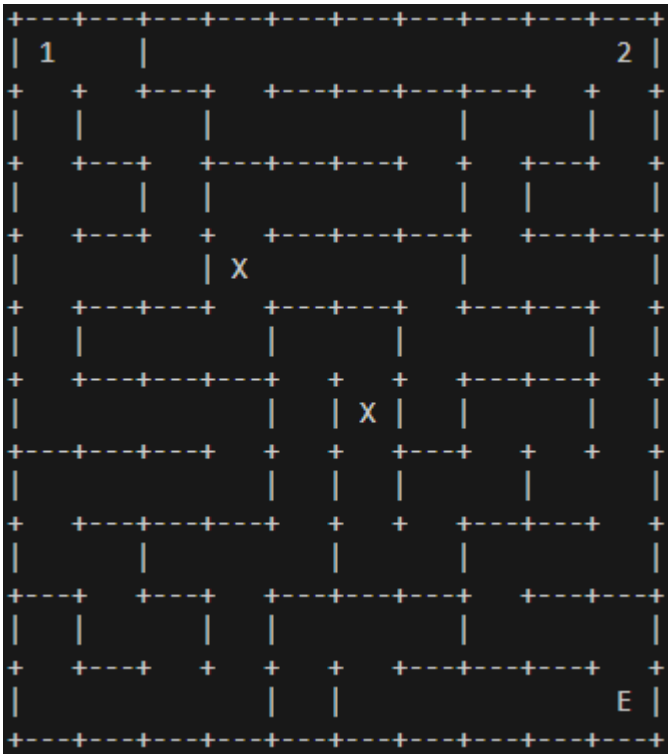
randomly generated maze



Exit message shown when the player successfully reaches the end of the maze.



Organized folder structure based on the example provided on Moodle.



Multiplayer maze view showing Player 1 and Player 2 starting at opposite corners.

```

72 String translatedDir = switch (input) {
73     case "I" -> "W";
74     case "K" -> "S";
75     case "J" -> "A";
76     case "L" -> "D";
77     default -> "";
78 };
79

```

Java switch expression used to translate IJKL inputs into WASD equivalents for Player 2 movement, enabling intuitive local multiplayer control.

resolve merge conflicts and finalize merge of coop into main



Carlos-0620 committed 17 hours ago

Merged beta into main: integrated mirrored maze generation and enhanced sprite logic with random movement

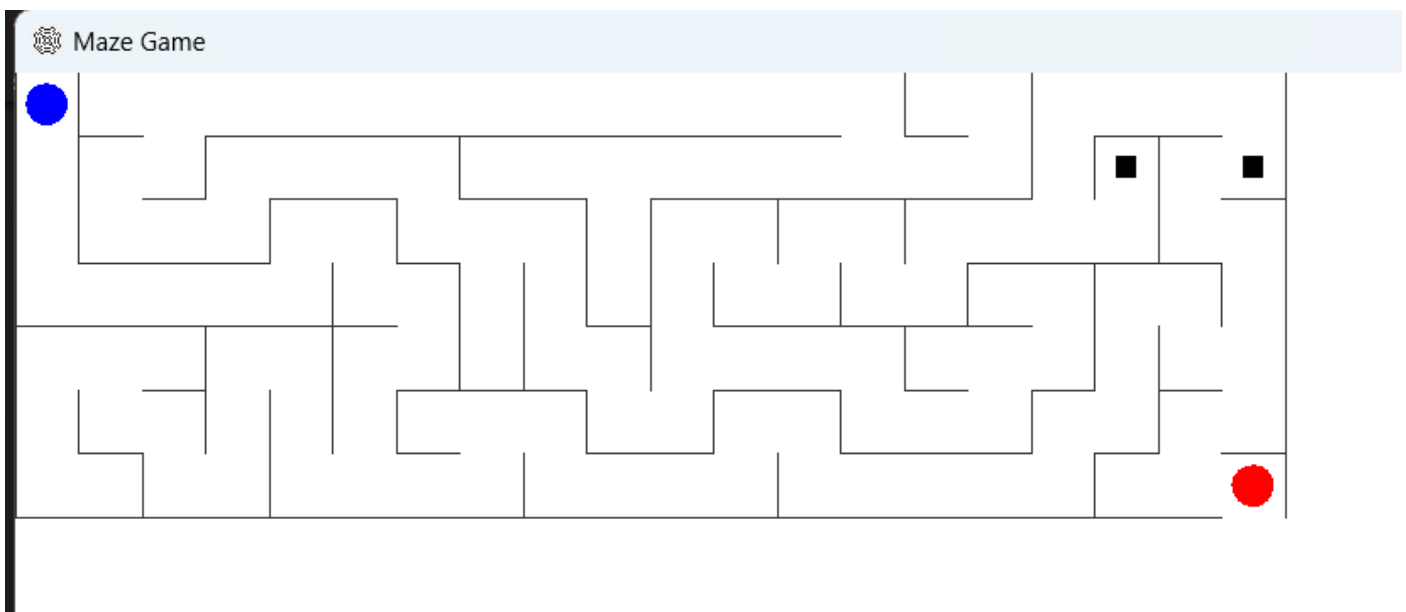


Carlos-0620 committed 17 hours ago

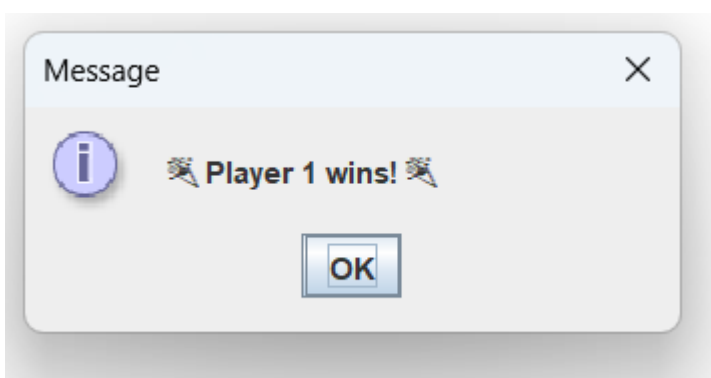
Commits showing Carlos's work on merging branches and integrating key gameplay features.



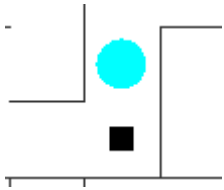
Player 1 starts in the top-left corner and Player 2 in the bottom-right.



graphical version of the maze



Winner message displayed when one of the players win



Change of color when colliding with a sprite resulting in freeze feature activated

Problems Encountered

During this iteration, I encountered two major technical issues that affected my workflow. First, my Visual Studio Code environment experienced conflicts between installed extensions, which prevented Java Swing windows from displaying properly. After multiple troubleshooting attempts, I resolved the issue by resetting the IDE to its default configuration, which restored window rendering functionality.

The second issue involved import errors while coding the graphical components. These were caused by missing or ambiguous library references, particularly when using classes like List and Timer, which exist in multiple Java packages. I resolved this by reviewing all import statements carefully and ensuring the correct packages (e.g., `javax.swing.Timer`, `java.util.List`) were used consistently throughout the project.

Despite these setbacks, both issues were successfully addressed, allowing me to continue development without further disruption.

Student 3: Georgii Taisaev

Georgii's Screenshots

1)

initialize basic game window structure
TgTstion • 20 hours ago

initialize basic game window structure

- Set up the initial GameWindow class with basic configurations.

2) • encapsulate window properties

- Improved structure by encapsulating window-related properties.

.

encapsulate window properties
TgTstion • 20 hours ago

3) integrate MazeGame into GameWindow

- Linked the game logic (MazeGame) with the graphical interface.

integrate MazeGame into GameWindow
TgTstion • 20 hours ago

4) clean up code and add comments

- Organized code and added explanatory comments.

integrate MazeGame into GameWindow
TgTstion • 20 hours ago

5) • Checking for collisions between players

- Introduced functionality to handle player collisions in the game.

.

Checking for collisions between playersc

TgTstion • 5 minutes ago

6)Checking for collisions between playersc

- Likely a refinement or correction related to the previous commit.

Checking for collisions between players



TgTstion • 4 minutes ago

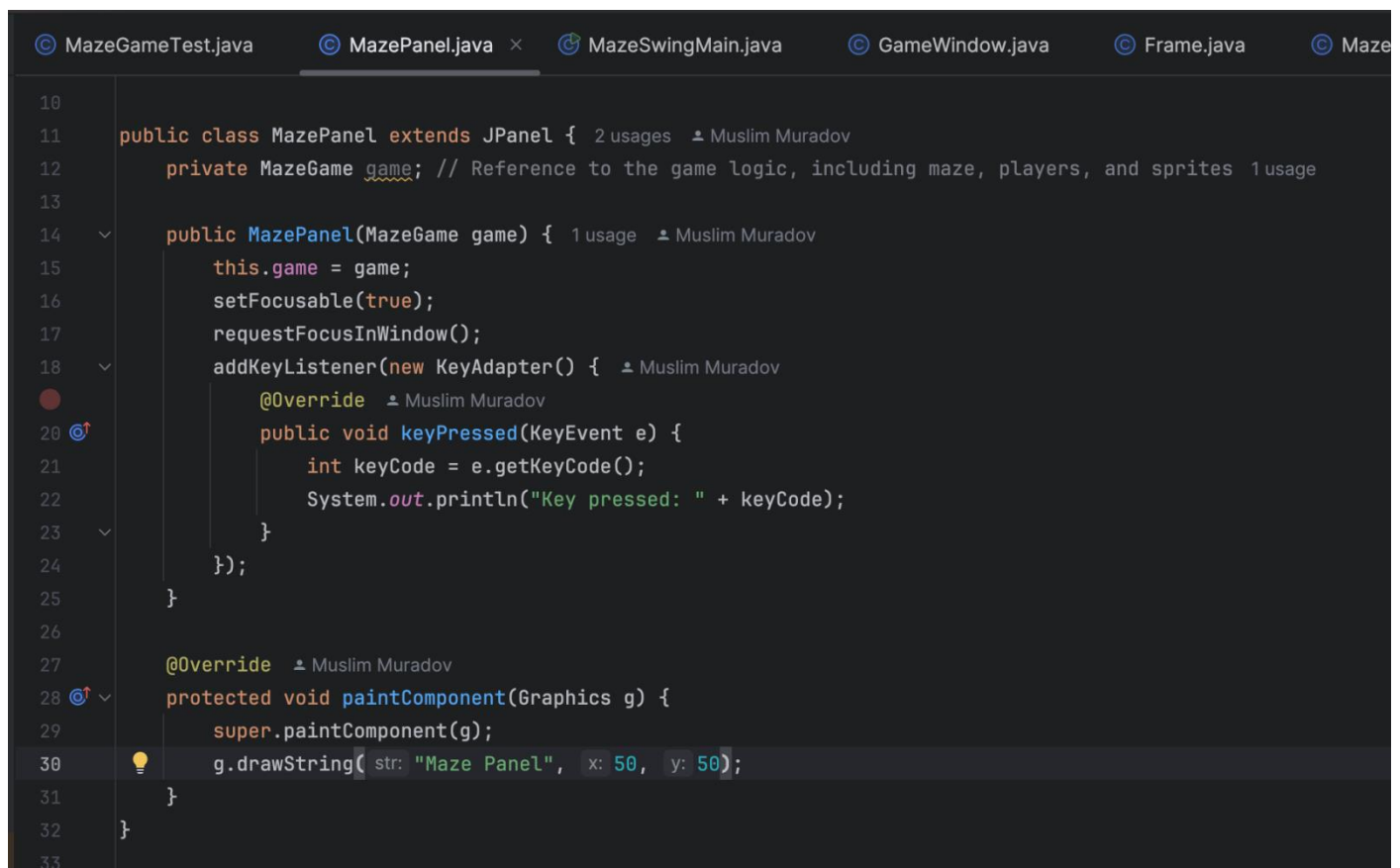
Student 4: Muslim Muradov - This is from report 2

In this part of the work, I integrated some graphical user interface (GUI) components into a maze game using Java Swing. This involved creating a MazePanel class to display a window and handling keyboard input to move the player. For now, I've closed this with placeholder so that I can later implement the maze itself and the players into the game window

Running the Swing application was difficult at first, I had to create a separate main class to run it. I kept the previous class so that the game could still be run in the console and tested. When we move the maze to Swing, we can get rid of the console version.

All in all, these additions laid the groundwork for further development of the game in a convenient window to ensure convenient gameplay and proper operation.

Muslim's Screenshots



```
10
11 public class MazePanel extends JPanel { 2 usages  Muslim Muradov
12     private MazeGame game; // Reference to the game logic, including maze, players, and sprites 1 usage
13
14     public MazePanel(MazeGame game) { 1 usage  Muslim Muradov
15         this.game = game;
16         setFocusable(true);
17         requestFocusInWindow();
18         addKeyListener(new KeyAdapter() {  Muslim Muradov
19             @Override  Muslim Muradov
20             public void keyPressed(KeyEvent e) {
21                 int keyCode = e.getKeyCode();
22                 System.out.println("Key pressed: " + keyCode);
23             }
24         });
25     }
26
27     @Override  Muslim Muradov
28     protected void paintComponent(Graphics g) {
29         super.paintComponent(g);
30         g.drawString(str: "Maze Panel", x: 50, y: 50);
31     }
32 }
33
```

```
MazeGameTest.java  MazePanel.java  MazeSwingMain.java  ×  GameWindow.java

1  package graphicalUserInterface;
2
3  import game.MazeGame;
4
5  //MazeSwingMain is a test class to launch the Swing version of the maze game.
6  public class MazeSwingMain {  Muslim Muradov
7      public static void main(String[] args) {  Muslim Muradov
8          // Create an instance of MazeGame
9          MazeGame game = new MazeGame();
10
11         // Create the main game frame passing the game logic
12         Frame frame = new Frame(game);
13
14         // Set the frame visible to start the GUI
15         frame.setVisible(true);
16     }
17 }
```

Added some changes in Frame class

```
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setResizable(true);
setSize(width: 1080, height: 720);
setLocationRelativeTo(null);
setVisible(true);

initializeGameUI();

MazePanel mazePanel = new MazePanel(game);
add(mazePanel);
```

Conclusion

The final version of the project is expected to deliver a fully functional graphical user interface (GUI) using Java Swing, replacing the previous console-based interaction. This enhancement will significantly improve the overall user experience by providing a more intuitive and visually engaging way to interact with the game. Rather than relying on ASCII characters and text-based prompts, the game will feature a visual rendering of the maze, player avatars, and moving sprites, offering a more modern and appealing interface.

One of the most important improvements will be enabling simultaneous multiplayer functionality. In the current version, both players take turns entering commands in the console. With the GUI, the goal is to support real-time keyboard input for both players, allowing them to move at the same time. This change will enhance gameplay fluidity and bring the experience closer to traditional cooperative or competitive games.

The visual design of the game will also be updated by introducing graphical representations for the sprites. Instead of using simple 'X' characters, sprites will be shown as unique images or icons on the screen. This will make the game visually clearer and more enjoyable, especially when sprites interact with players, such as during collisions or when players are caught.

Another planned improvement is the addition of a basic main menu. This menu would allow players to start a new game, configure maze dimensions, or exit the application. Including customizable settings such as difficulty level or sprite behaviour will give users more control over their experience and accommodate different skill levels.

Finally, the game will integrate visual and possibly audio feedback to better communicate events to the player. For instance, when a collision occurs, the game could highlight the affected cell or trigger a sound effect to make the event more noticeable. These changes are intended to create a polished, user-friendly, and engaging game environment in the final version.

Link to our repository:

https://github.com/Carlos-o620/Software_Development_Project.git

References

1. Midway Manufacturing Co. (1976). Amazing Maze [Arcade Game]. International Arcade Museum. Retrieved from <https://www.arcade-museum.com/Videogame/amazing-maze>
2. Atari. (1980). Maze Craze [Atari 2600 Game]. Atari Official Website. Retrieved from <https://atari.com/pages/mazecraze>
3. Buck, J. (2015). Mazes for Programmers: Code Your Own Twisty Little Passages. The Pragmatic Bookshelf. ISBN: 978-1-68050-055-4
4. GeeksforGeeks. (n.d.). Introduction to Java Swing. Retrieved from <https://www.geeksforgeeks.org/introduction-to-java-swing/>
5. GeeksforGeeks. (2022). Switch statement in Java. Retrieved from <https://www.geeksforgeeks.org/switch-statement-in-java/>
6. JUnit Team. (n.d.). JUnit 5 User Guide. Retrieved from <https://junit.org/junit5/docs/current/user-guide/>
7. GitHub Docs. (n.d.). About merge conflicts. Retrieved from <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts/about-merge-conflicts>